



Izan de Vega López
UCM - Desarrollo de Videojuegos
Simulación física para videojuegos

Resumen

Galactic Armada es un juego de naves espaciales en el que controlarás tu propia nave y tu objetivo será derribar a todas las naves enemigas. Se usa la librería de physx y múltiples aproximaciones a efectos físicos reales para obtener una mayor inmersión e interactividad con el entorno.

Resumen	1
Enlace al repositorio:	2
Enumeración de los efectos incorporados	2
Requisitos del proyecto implementados:	2
Dos generadores de partículas distintos:	2
Gestión de creación y destrucción de instancias (las partículas no pueden estar indefinidamente en escena, teniendo límites espaciotemporales de existencia):	3
Destrucción de todos los elementos al salir de la escena.	3
Partículas con distinta masa.	3
Un sistema de sólidos rígidos.	3
Sólidos rígidos con distinto tamaño, forma, masa y tensor de inercia.	3
Dos generadores de fuerzas diferentes (sin contar muelles). cada uno con su fórmula y sus restricciones de aplicación.	4
Interacción con el usuario a través del teclado y/o ratón.	4
Un ejemplo de muelle o de flotación.	4
Diagrama de clase del sistema de partículas y de sólido rígido empleado.	5
Sistemas de partículas:	5
Sólido-Rígidos:	5
Ecuaciones físicas utilizadas	6
Bomba:	6
Muelles:	6
Viento:	6
Integración:	6
Manual de usuario, contenido en el README.md (descripción somera y función de cada tecla o modo de interaccionar con el videojuego).	7
Experimentos extra	7
Escenas por GameObjects	7
Jerarquía de objetos y objetos compuestos:	7
Creación de generadores de partículas en el punto de impacto de un proyectil del jugador en una nave enemiga. Conservando posición y rotación conforme la nave se mueve y rota:	7
Input de ratón y mejora del input existente de teclado.	8
Obtención del vector de torque para que las naves enemigas apunten al jugador y para que la nave del jugador rote según el movimiento del ratón.	8
Render de imágenes 2D, menús y pantalla de victoria.	9
Modificación de la cámara para que tenga en cuenta que dirección es arriba:	9
Movimiento de cámara	10

Enlace al repositorio:

<https://github.com/Eric11195/SimulacionFisicaVideojuegos>

Enumeración de los efectos incorporados

Aquí se enumeran sólo los efectos que forman parte de los requisitos obligatorios, para efectos adicionales consultar la sección [experimentos extra](#).

- El usuario puede interactuar con el programa con teclado y ratón, con el efecto de controlar una nave y lanzando distintos tipos de proyectiles desde su posición.
- Movimiento de naves con propulsores (generadores de fuerza) activables. Rotan junto con las naves, de forma que siempre les dan fuerza en la dirección en la que están mirando.
- Disparo de parejas de partículas unidas entre sí por un muelle.
- Modelado de una explosión al morir una nave.
- Movimiento de entidades en la escena con el método de integración de Euler semi implícito
- Generación de partículas con parámetros configurables y aleatorizados para los propulsores de naves, estela de un misil, explosiones y distintos tipos de proyectiles
- Uso de sólidos rígidos mediante la librería physx con colisiones entre ellos para mostrar las colisiones entre naves, o entre proyectiles y naves enemigas.

Requisitos del proyecto implementados:

Dos generadores de partículas distintos:

En el proyecto hay multitud de generadores de partículas.

La clase base se puede encontrar en el fichero [ParticleGenerator.cpp](#) y [.hpp](#), junto con varias subclases. Entre ellas, generadores on/off (ToggleParticleGenerator), activables instantáneamente con una función (TriggeredParticleGenerator) o que aplican fuerzas a sus partículas (ForceAffectedParticleGenerator).

Las partículas tienen múltiples parámetros que definen sus comportamientos. El archivo de configuración que define estos parámetros para las partículas modelo de los generadores es [ParticleDescriptor.cpp](#) y [.hpp](#). Y los modificadores (el uso de las distribuciones aleatorias para generar las partículas con ciertas desviaciones respecto a los parámetros de la partícula modelo) en [ParticleGeneratorsDescriptors.cpp](#) y [.hpp](#).

El proyecto cuenta además con generadores de partículas algo más complejos, usados para los proyectiles de la nave.

Estos tres primeros tienen la particularidad de que generan partículas con colisiones físicas gracias a physx. Además al dispararse desde la nave se les otorga la velocidad a la que fuese la nave como velocidad adicional inicial.

- [SpringJoinedParticleLauncher.cpp](#) [.hpp](#) Genera 2 partículas unidas por un muelle (Clic derecho del ratón).
- [ShipRegularProjectileCannon.cpp](#) [.hpp](#) Lanza un único proyectil de physx (Clic izquierdo del ratón).
- [MissileGenerator.cpp](#) [.hpp](#) Lanza un misil que tiene su propio generador de partículas para su estela (Tecla espacio).
- [BombGenerator.cpp](#) [.hpp](#) Se usa cuando muere una nave enemiga para generar una explosión. Este genera partículas sin colisiones ni objetos de physx.

Gestión de creación y destrucción de instancias (las partículas no pueden estar indefinidamente en escena, teniendo límites espaciotemporales de existencia):

Por la arquitectura de GameObjects que uso, todos los objetos de la escena tienen una función virtual `alive` que devuelve si deberían ser borrados en el `step` (lo que me permite borrar naves enemigas derribadas fácilmente).

Para las partículas, esta función devuelve si su tiempo de vida restante es mayor que 0, y cada frame (en su `step`) se resta el tiempo entre frames a su tiempo de vida restante. Esto se puede ver en el `Particle::step` en la línea 11 de [Particle.cpp](#). Y en el `Particle::alive` en la línea 27 de [Particle.hpp](#)

Además en el caso de los objetos generados por un generador de partículas. Una función `inside_area_of_interest` permite saber si las partículas se han salido de los límites que tienen establecidos, y de ser así poder borrarlas prematuramente (línea 115 de [ParticleGenerator.cpp](#)).

Este método recibe una función lambda del archivo [ParticleGeneratorsDescriptors.cpp](#). Es decir, cada generador de partículas define particularmente (no pun intended) cuál es esta función en su caso. Podemos ver un ejemplo de una función definida en la línea 250, que define el rango máximo del misil.

Destrucción de todos los elementos al salir de la escena.

Al salir del juego se destruyen todos las escenas (que son `gameobjects`), borrando todo lo que se hubiese creado dentro de cada una (los `game objects` hijos que contienen). Esto se puede ver en el `cleanupPhysics` (línea 217) de [main.cpp](#).

Partículas con distinta masa.

En [ParticleDescriptor.cpp](#) y [.hpp](#) se definen distintos tipos de partículas, cada una con una posible masa distinta. Como se puede comprobar al disparar a las naves enemigas con distintos proyectiles las reacciones tienen distintos niveles de fuerza. Pues aparte de distintas velocidades cuentan además con masas diversas.

Por ejemplo, en la línea 59 vemos que las partículas generadas al explotar una nave enemiga tienen una masa de 0.1kg. Y en la línea 223 que las partículas unidas por un muelle tienen cada una una masa de 40 kg.

Un sistema de sólidos rígidos.

Se ha creado la clase [RigidbodyObject.cpp](#) y [.hpp](#). Que contiene un puntero a un objeto dinámico de `physx` asociado. Esta clase vuelve a heredar de `GameObject` y sobrescribe gran parte de las funciones usadas hasta ahora para que sigan teniendo la misma funcionalidad pero aplicada a los objetos rígidos dinámicos de `physx`.

También existe `StaticRigidbody_Object` (línea 47 en [RigidbodyObject.hpp](#)) que hace lo mismo pero para objetos estáticos de `physx`. Esta segunda clase no es usada en el resultado final del proyecto.

Estas clases permiten opcionalmente darles una representación visual. Esto es así para poder tener formas complejas y hacer una caja de colisión simplificada invisible. Como es el caso de las naves enemigas formadas por varios objetos pero que solo usan un cubo invisible para su colisión.

Sólidos rígidos con distinto tamaño, forma, masa y tensor de inercia.

Cada proyectil disparado por la nave es un sólido rígido con el tamaño del proyectil y forma esférica. Las naves usan sólido-rígidos invisibles con forma de cubo para sus colisiones.

Los tres generadores de partículas de [este](#) apartado generaban objetos sólido-rígidos. Se puede ver en esos mismos ficheros como se invoca al constructor de una de las clases hijas de

[RigidbodyObject.cpp](#). Estas clases (visibles en ese mismo fichero) simplemente permiten generar cubos (Rigid_CubeObject) o esferas (Rigid_SphereObject) de forma rápida.

Para un ejemplo de objetos con forma de cubo podemos ver [Ship.cpp .hpp](#) o [EnemyShip.cpp .hpp](#). Ambas heredan de Rigid_CubeObject y como el resto de objetos del juego desactivan la gravedad en su creación al ser un juego en el espacio donde queremos ser capaces de poder movernos de forma libre.

Las naves definen su masa al inicio de los ficheros [Ship.cpp](#) y [EnemyShip.cpp](#) correspondientemente. Que finalmente será usada en updateMassAndInertia como densidad del objeto para obtener el tensor de inercia. Aunque siendo las naves objetos de 1x1x1 de tamaño acaba siendo su masa igualmente.

Los proyectiles usan Rigid_SphereObject, (siendo el resultado esferas) y obtienen su masa y radio del archivo de configuración [ParticleDescriptors.cpp](#) al igual que lo hacían las partículas generadas con el resto de generadores. Los 3 generadores mencionados en [este](#) apartado los generan llamando a sus constructores con la masa y radio. Aunque la masa vuelve a ser usada como densidad aquí. Eso sí, aquí no es equivalente pues el volumen de una esfera es $4 \cdot \pi \cdot \text{radio}^3$. Así que la masa total acaba siendo menor del parámetro usado en la mayoría de los casos. Al ser todos los proyectiles de radio bastante inferior a 1.

Siendo el resultado del producto de nuestra masa(que realmente es densidad) * $4 \cdot \pi$ * radio la masa real final.

Dos generadores de fuerzas diferentes (sin contar muelles), cada uno con su fórmula y sus restricciones de aplicación.

El misil disparado con el espacio tiene un generador de fuerzas de dirección contraria a la dirección en la que avanza modelado con la fuerza del aire (como se expone en el apartado de [formulas físicas utilizadas](#)), para conseguir que las partículas que genera salgan disparadas hacia atrás, como lanzadas por la fuerza de repulsión trasera del cohete. Esto se puede ver en [MissileGenerator.cpp](#) línea 37.

Las naves cuentan con sus propulsores que son generadores de fuerza direccionales simples que se pueden activar a voluntad [Ship.cpp](#) líneas 28-31. Estos simplemente se rotan en función de la orientación actual de la nave para que otorguen fuerza en la dirección en la que mira la nave.

Las naves enemigas explotan al morir haciendo uso de un generador de fuerza de explosión, que hace uso de la fórmula en este [apartado](#).

Interacción con el usuario a través del teclado y/o ratón.

Los gameobjects heredan de input processor (en [InputProcessor.cpp](#)). Y cada vez que un input se produce se propaga a todos los gameobjects para que cada uno reaccione como sea necesario. Así, la nave recibe inputs tanto de teclado como de ratón para su movimiento y disparos.

Los botones del menú principal y de victoria también usan este input para detectar si el ratón está sobre ellos y activarse.

Un ejemplo de muelle o de flotación.

Con clic derecho la nave lanza dos proyectiles que están unidos por un muelle. La explicación de la fórmula y código que lleva a cabo este comportamiento se puede ver en el apartado [Muelle](#) de Ecuaciones Físicas utilizadas.

Diagrama de clase del sistema de partículas y de sólido rígido empleado.

He añadido el diagrama al repositorio de github. Se puede ver [aquí](#).

Tanto los sistemas de partículas como los sólidos rígidos son gameobjects, al igual que todos los elementos en mi escena. Un gameobject tiene una lista de gameobjects que son hijos suyos. Un gameobject se puede usar como una escena. De forma que la imagen global de la escena sería un árbol de gameobjects que lleva implícitas sus jerarquías.

Sistemas de partículas:

Al hacer el step de un gameobject se hace también el step de todos sus hijos, al renderizar un gameobject se renderizan también todos sus hijos, como se puede ver en el step de [GameObject.cpp](#) y la mayoría de sus otros métodos. Ver el apartado experimentos extra para más explicaciones.

Los sistemas de partículas en mi arquitectura realmente se pueden sustituir por un gameobject plano. Pues lo único que hacen es llamar al step de los hijos. La lógica pesada realmente está en los generadores de partículas ([ParticleGenerators.cpp](#) [.hpp](#)). Pues estos cada frame hacen el step de todas las partículas que contienen, generan nuevas partículas y comprueban si están dentro de los límites definidos para borrarlas en caso contrario. Esto se puede ver en el step de la línea 59 del [.cpp](#)

Estos mismos generadores contienen funciones lambda obtenidas del archivo [ParticleGeneratorDescriptor.cpp](#). Que permiten alterar individualmente los distintos parámetros de las partículas que generan e introducir aleatoriedad en los parámetros: masa, posición, velocidad, dirección, tiempo de vida y color de los objetos generados.

Además, estos generadores pueden generar cualquier otro objeto que herede de gameobject, de forma trivial si heredan de RigidParticle o de Particle. O cambiando un par de líneas de código más en caso contrario. Pero realmente permiten un gran nivel de variabilidad en el tipo y comportamiento de objeto generado con pocos cambios.

Estos generadores de partículas además contienen una lista con todos los generadores de fuerza que se han de aplicar a las partículas en su interior. De forma que al crearse se suscriban a estos generadores para recibir fuerza de ellos. Esto es visible en ForceAffected_ParticleGenerator::set_up_particle de la línea 32 del [.cpp](#)

Sólido-Rígidos:

Mis [RigidbodyObject.cpp](#) y [.hpp](#) heredan de SceneObject, un GameObject con representación física, y contienen en su interior un puntero a un objeto de tipo physx::PxRigidDynamic. En su constructor reciben un puntero a una physx::PxShape que comparten con el renderItem contenido en SceneObject.

Al ser gameobjects se renderizan y actualizan (hacer su función step) de la misma manera que todo el resto de objetos de la escena. Lo único que hacen es hacer override de todos los métodos que usaba GameObject para adaptarlos a los objetos de physx como se puede ver en el [RigidbodyObject.cpp](#). Entre otras modificaciones, modifican el integrate para que solo sume todas las fuerzas aplicadas al objeto y añada la fuerza correspondiente con la llamada al método del PxRigidDynamic y cada step actualizan la posición y rotación del RenderItem con la del PxRigidDynamic para que se corresponda lo visto con los cálculos de la librería.

Ecuaciones físicas utilizadas

Bomba:

Cuando una nave es impactada por un proyectil la nave explotará breve tiempo después. [BombGenerator.cpp .hpp](#).

Esta es la fórmula usada en la línea 17 en adelante:

K es una constante que mide la magnitud de la fuerza.

r es la distancia hasta el centro de la explosión. Y R el radio dentro del que se aplica la fuerza de la explosión.

t es el tiempo

Y tao es otra constante que define cómo evolucionará la fuerza aplicada por la explosión en el tiempo.

Se pueden ver los parámetros que se les asignan a K, R y tao (en ese orden) en la línea 146 de [EnemyShip.cpp](#).

$$\vec{F}_e(x, y, t) = \begin{cases} \frac{K}{r^2} \begin{bmatrix} x - x_c \\ y - y_c \\ z - z_c \end{bmatrix} e^{-\frac{t}{\tau}} & \text{si } r < R \\ \vec{0} & \text{e.o.c} \end{cases}$$

Muelles:

Al pulsar el clic derecho se disparan dos partículas unidas por un muelle.

La siguiente es la fórmula que se usa para modelar este comportamiento. Esto se puede ver en [ForceGenerator.cpp](#) en la función SpringForceGenerator::calculate_force() línea 178 del cpp.

Creamos estas partículas y el generador de fuerza para cada par de partículas en [SpringJoinedProjectileLauncher.cpp](#).

$$\vec{F} = -k(|\vec{d}| - l_0) \cdot \left(\frac{\vec{d}}{|\vec{d}|} \right)$$

Siendo k una constante de la magnitud de la fuerza.

d es el vector de la longitud actual del muelle.

l₀ es la longitud del muelle en reposo.

Viento:

La fuerza que genera el misil disparado con la tecla espacio y que se les otorga a las partículas que salen por su retaguardia está modelado como la fuerza del viento.

Podemos ver la fórmula usada en [ForceGenerator.cpp](#) en la línea 83. En la función Wind_ForceGenerator::calculate_force.

$$\vec{F}_v = k_1(\vec{v}_v - \vec{v}) + k_2\|\vec{v}_v - \vec{v}\|(\vec{v}_v - \vec{v})$$

Siendo F_v la fuerza del viento final. v_v es la velocidad del viento, v la velocidad de la partícula. Y k₁ y k₂ el coeficiente de rozamiento con el aire. k₂ es usado para altas velocidades de viento, cuando el flujo se vuelve turbulento. En nuestra simulación k₁ tiene un valor de 1 y k₂ un valor de 0 al ser la velocidad del viento reducida.

Integración:

Usamos la integración por euler semi implícita. En [GameObject.cpp](#) GameObject::integrate línea 114 del cpp. Primero calculamos la aceleración con las fuerzas aplicadas, luego sumamos la aceleración a la velocidad y por último cambiamos la posición del objeto. Además multiplicamos la velocidad por el valor de dumping para reducir los errores de precisión.

Manual de usuario, contenido en el README.md (descripción somera y función de cada tecla o modo de interaccionar con el videojuego).

Enlace al [README.md](#)

Experimentos extra

Escenas por GameObjects

Mi arquitectura está basada en gameobjects que contienen a otros gameobjects. Y al hacer el step de uno, se hace el step de todos los que contiene recursivamente. Esto, aparte de permitirme tener jerarquías de objetos como explico en el siguiente apartado, me permite tener un game object por escena en mi juego y con solo decir que game object es mi escena actual esto hace que se actualicen, se rendericen y reciban el input solo los gameobjects hijos del gameobject elegido.

Por lo que he movido a otros ficheros o cambiado las clases de render dadas inicialmente en el proyecto para que se adapten a este sistema.

Jerarquía de objetos y objetos compuestos:

La arquitectura de GameObjects ([GameObject.cpp .hpp](#)), en la que todos los objetos de la escena heredan de esta clase, me permite tener una jerarquía de objetos padres e hijos, gracias a que cada GameObject tiene una lista de punteros a GameObjects que son sus hijos. Y gracias a esto puedo obtener fácilmente transformaciones relativas al objeto padre.

En mi escena esto se manifiesta en tener objetos como los cazas TIE, una nave con cañones que apuntan en la dirección hacia la que se orienta la nave y generadores de partículas o fuerza que rotan con el objeto que los contiene.

La lógica para conseguir esto es en realidad sencilla gracias a la implementación de vectores y quaternions de physx.

Un objeto padre puede obtener la posición de sus objetos hijos a partir de la suya de la siguiente manera:

```
physx::PxVec3 Posición_final_hijo =  
Quaternion_padre.rotate(Posición_relativa_del_hijo_con_respecto_al_padre);  
physx::PxQuat Rotación_final_hijo = Quaternion_padre * Quaternion_hijo;
```

Esto se puede ver por ejemplo en el step de las naves enemigas, donde se van colocando todos los objetos que las forman en las posiciones y con las orientaciones oportunas ([EnemyShip.cpp](#) líneas 88-128).

Creación de generadores de partículas en el punto de impacto de un proyectil del jugador en una nave enemiga. Conservando posición y rotación conforme la nave se mueve y rota:

Eso se consigue gracias a la implementación de los objetos compuestos del apartado anterior.

Cuando un proyectil impacta a la nave physx nos dará la posición en espacio global de la colisión. Lo que tenemos que conseguir es esa posición con respecto a la nave enemiga. Y obtener la dirección relativa en la que deberían salir proyectadas las partículas. Una vez obtenidas estas dos cosas podemos añadir el generador de partículas como hijo de la nave y usar las fórmulas anteriores para obtener su posición y orientación en cada step.

Los siguientes cálculos se pueden encontrar en EnemyShip::set_collision_point, línea 137 de [EnemyShip.cpp](#)

Lo primero que haremos será obtener el quaternion que nos permite cambiar de posición global a posición padre. Este quaternion es el conjugado del quaternion que tiene el objeto padre en su transform:

```
PxQuat global_pos_to_this_obj_pos =  
    global_transform.q.getConjugate();
```

Con este podemos obtener la posición relativa:

```
generador_fuego_pos_relativa =  
    global_pos_to_this_obj_pos.rotate(colision.position -  
    global_transform.p);
```

Obtener la orientación es algo más complejo. Vamos a obtener el quaternion de orientación relativa respecto al padre a partir del par de rotación. Para esto necesitamos el ángulo y el vector a partir del cual se efectuará la rotación.

El ángulo que necesitamos es el que forman la posición relativa del generador con respecto a la dirección en la que mira el padre, que es {0,0,1}, entonces el ángulo en radianes sería el siguiente, pues el producto escalar de dos vectores normalizados nos da el coseno del ángulo entre ambos:

```
PxVec3 pos_normalized =  
    generador_fuego_pos_relativa.getNormalized();  
float angle = physx::PxAcos(pos_normalized.dot({ 0,0,1 }));
```

Y por último queremos el vector perpendicular a los dos anteriores, con el que podremos saber el eje respecto al que se efectuará la rotación. Y ya podemos obtener el quaternion que buscábamos:

```
PxVec3 perpendicular =  
    physx::PxVec3(0, 0, 1).cross(pos_normalized).getNormalized();  
PxQuat Quaternion_relativo = PxQuat(angle, perpendicular);
```

Con estas dos cosas tenemos ya el Transform relativo respecto al padre. Con el que podremos actualizar la posición y orientación del generador de partículas para obtener el efecto deseado con las ecuaciones del anterior apartado

Input de ratón y mejora del input existente de teclado.

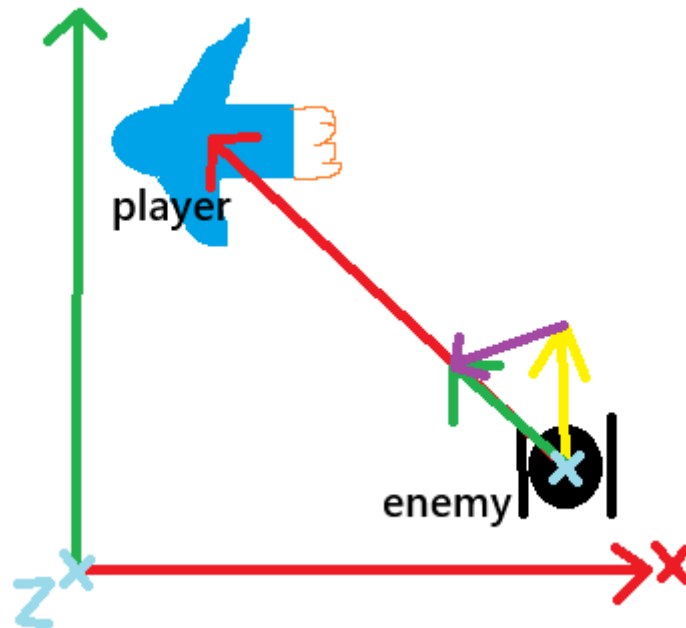
Ahora también se detecta la posición del ratón en pantalla y los botones del ratón. Se han añadido en [main.cpp](#) las funciones mousePressed, mouseReleased y mousePosUpdate para este propósito.

Además ahora se detecta cuando una tecla se pulsa y cuando se deja de pulsar (métodos keyPressed y keyReleased de [main.cpp](#)), en vez de si está pulsada. Esto permite evitar que las teclas funcionen igual que cuando escribes. Caso en el que primero escriben una letra, dejan medio segundo de espera y después la repiten muchas veces si sigue estando pulsada. Al evitar esto el control se siente más fluido.

Se han modificado o añadido las funciones motionCallback, mouseCallback, keyboardCallback y keyboardUpCallback en [RenderUtils.cpp](#)

Obtención del vector de torque para que las naves enemigas apunten al jugador y para que la nave del jugador rote según el movimiento del ratón.

Para que la nave enemiga rote para mirar al player necesitamos una velocidad y el eje de giro. Podemos usar un diagrama para verlo mejor.



El eje de giro podemos obtenerlo con el vector perpendicular al vector que desde la nave enemiga apunta al player y al vector dirección de la nave enemiga (amarillo). Suponiendo que ambas naves están en la misma coordenada z este vector se corresponde con el vector $(0,0,1)$. Representado con una cruz cian en la imagen.

Y la velocidad de giro podemos obtenerla con la magnitud de la resta de los dos anteriores vectores normalizados(amarillo y verde que salen de la nave), este último es el vector morado en el diagrama.

Rotar la nave del jugador es mucho más sencillo, como se puede ver en el step (línea 39) de [Ship.cpp](#) (claro que la recepción del input y procesado para convertir la posición del ratón a un torque ha sido calculado previamente en el método `Ship::handle_mouse_pos` línea 123).

Hacer roll (teclas A y D) se corresponde con añadir torque en el eje z (que es la dirección en la que mira la nave). La posición del ratón en x se corresponde con añadir torque en el eje y, al contrario que para el eje y de la pantalla que habría que añadir torque en el x.

A estos vectores unitarios de dirección solo habría que multiplicarlos por un número para que la rotación fuese más lenta o más rápida. En el caso de las rotaciones con el ratón, este número es más grande cuanto más lejos del centro de la pantalla se esté (limitado a un máximo y de forma porcentual para que al ampliar la pantalla siga funcionando de forma equivalente).

Render de imágenes 2D, menús y pantalla de victoria.

Se ha añadido un sistema de renderizado de imágenes 2D (visible en [hud_elem.cpp](#)) y una librería de carga de imágenes([stb_image.hpp](#)).

Combinado con el input de ratón esto permite tener botones ([buttons.cpp .hpp](#)) en pantalla. Y una imagen transparente que haga de casco de la nave por ejemplo. Los botones reciben una función en su creación que permite definir su comportamiento al ser pulsados. De esta forma pueden cambiar de escena o cerrar el juego.

Modificación de la cámara para que tenga en cuenta que dirección es arriba:

He añadido un parámetro up en la cámara que sustituye lo que en la implementación inicial era una constante. Esto permite el movimiento de la cámara junto con la nave y que al renderizar la escena

se corresponda la vista de la nave con lo que debería estar viendo en caso de que su dirección arriba no se corresponda con el vector $(0,1,0)$.

Movimiento de cámara

El transform de la cámara se actualiza cada frame con respecto al transform de la nave del jugador como si fuese un gameobject hijo más. Esto ocurre en [Ship.cpp](#) línea 158, en la primera línea de la función Ship::update_child_transforms

Disclaimer:

Las imágenes usadas en los menús del proyecto o como casco de la nave han sido generadas con IA generativa o sacadas de internet. Se pueden ver estas imágenes en la carpeta [skeleton del proyecto](#).

No se ha usado IA en ningún aspecto más del proyecto.